

# Spark's Sorting Strategies

---

Assuming the question is asking about the different ways to use sort and join in relation to Dataframes, there are 3 layers to the answers.

First, Spark exposes 3 main sorting functions: `.sort()`, `.orderBy()`, & `.sortWithinPartitions()`. For these, `.sort()` and `.orderBy()` are equivalent, and are used interchangeably based on readability. `.sortWithinPartitions()` however, is different from the other two.

This is where the second layer comes in, where these 3 sort functions, and really all the sorting operations in Spark differ. `.sortWithinPartitions()` can be said to be a "local" or In-memory sort. When called, Spark will sort the records that already are within the defined partition's allocated memory. This is similar to the behavior when a local sort occurs. The key part here is that this sort only resolves *local order*, whilst both `.sort()` and `.orderBy()` are able to operate on a global scale by partitioning. These sort functions shuffle the data into contiguous key ranges, and then has the lower level ( `.sortWithinPartitions()` ) sort handle the sorting within these partitions. Because the global sort handles the partitioning boundaries, it results in a set of partitions which are individually sorted and are ordered relative to one another. It is worth noting that Spark can also handle scenarios where the data in a partition exceeds the allocated memory of the executor. In this scenario, the sort spills-to-disk and operations occur on data on disk and in-partition.

The third layer is the interaction of sorting with joins. This is commonly referred to as "Sort-Merge Join" (SMJ), and involves the shuffling of both sides so that matching keys in both datasets are in the same partition. Then they are sorted locally and once completed, they are merged in a linear pass. For large-large joins, this is the default for Spark. There also are non-sort joins, "Broadcast Hash Join" (BHJ) and "Shuffle Hash Join" (SHJ), for which BHJ becomes relevant when there is a mismatch in sizes between the sides.

## Appropriate Dataframe Sorting Strategy for joining Parquet File 1 and 2

---

Now, for our scenario, the best choice would be to do **Broadcast Hash Join** where Dataset B is broadcast to the executing partitions of Dataset A.

1. Dataset B is relatively small. It is ~10,000 rows, 2 small columns, and not expected to change much. Thus, it is small enough to be under the broadcast threshold, and small enough to reasonably duplicate into every partition's memory.
1. Dataset A is large. With ~1,000,000 rows and several columns. If we chose Sort-Merge Join, we would have to shuffle and sort all the rows, just to match the keys. This is not efficient and does not provide any benefit as a local hash-table lookup resolves the match.
1. By choosing BHJ, we avoid the need to shuffle or sort Dataset A, reducing the number of shuffle stages required in comparison to SMJ.

On a code level, Spark automatically performs BHJ based on the defined broadcast threshold, but it is best practice to explicitly request broadcast to protect against scenarios where the broadcast dataset crossed the threshold silently.

```
from pyspark.sql.functions import broadcast

result_df = dataset_a_df.join(
    broadcast(dataset_b_df),
    on="geographical_location_oid",
    how="inner",
)
```